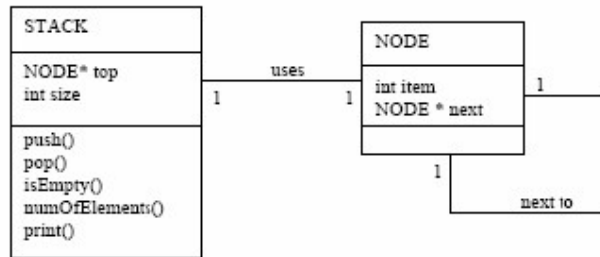


## BÀI THỰC HÀNH SỐ 2

**Bài 1:** Cho thiết kế cấu trúc ngăn xếp (xem định nghĩa và ví dụ ở cuối bài) dành cho các số nguyên dưới đây. Từ điển dữ liệu dưới còn thiếu điều kiện trước và điều kiện sau cho mỗi phương thức, hãy bổ sung để hoàn chỉnh bản thiết kế.



**Node:** một nút trong danh sách liên kết, chứa hai thành viên dữ liệu

- item: dữ liệu kiểu int
- next: con trỏ tới nút tiếp theo trong danh sách (có giá trị null nếu nút nằm cuối danh sách (gợi ý, có thể dùng cấu trúc struct))

**Stack:** ngăn xếp cho dữ liệu kiểu int,

*Các thành viên dữ liệu:*

- top: con trỏ tới đỉnh ngăn xếp (có giá trị null nếu ngăn xếp rỗng)
- size: số phần tử dữ liệu hiện có mặt trong ngăn xếp

*Phương thức:*

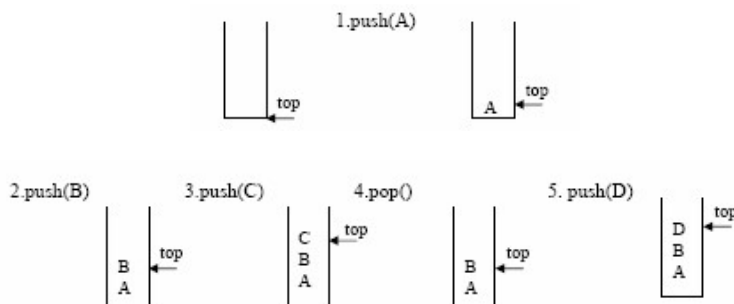
- push(): đẩy một phần tử dữ liệu vào đỉnh ngăn xếp, tham số là một giá trị kiểu int (giá trị của phần tử cần tiếp nhận), không trả về giá trị gì
- pop(): nếu ngăn xếp không rỗng, lấy phần tử trên đỉnh ngăn xếp ra khỏi ngăn xếp, trả về giá trị của phần tử đó; nếu ngăn xếp rỗng, trả về giá trị -1
- isEmpty(): trả về giá trị TRUE nếu ngăn xếp rỗng, ngược lại, trả về FALSE
- numOfElement(): trả về số phần tử hiện có trong ngăn xếp.
- print(): in ra danh sách các phần tử nằm trong stack, bắt đầu từ đỉnh stack

2. Cài đặt thiết kế trên bằng C++. Chú ý: phải có chú thích đầy đủ.

3. Viết một chương trình nhỏ để test sản phẩm của câu 2. Gợi ý: tạo một đối tượng stack, thử nghiệm các phương thức của đối tượng khi stack có 0, 1, 2, 3 phần tử.

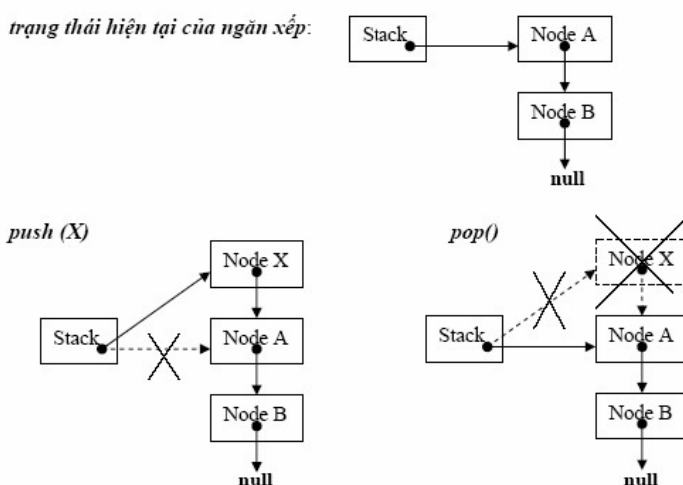
**Định nghĩa:** Ngăn xếp là cấu trúc dữ liệu Last-in-First-out, dữ liệu được đưa vào tại đỉnh ngăn xếp (thao tác push) và lấy ra từ đỉnh ngăn xếp (thao tác pop).

**Ví dụ:** Ban đầu, ngăn xếp rỗng. Sau đó, thêm các phần tử vào ngăn xếp



## Gợi ý về hoạt động của push và pop

trạng thái hiện tại của ngăn xếp:



## Bài 2:

- ❖ Sửa lớp Node trong câu 2.1 như sau:
  - a. bổ sung thành viên dữ liệu *id* (identification number). Mỗi khi một thể hiện mới của Node được tạo, giá trị cho *id* được sinh tự động theo cơ chế tăng dần (tương tự kiểu auto number của MSAccess) bắt đầu từ 1.
  - b. bổ sung phương thức print() để in ra nội dung của Node gồm *id* và *item* nằm trên 1 dòng, tách nhau bằng 1 dấu tab.
  - c. bổ sung copy constructor cho Node.
- ❖ Sửa lớp Stack trong câu 2.1 như sau:
  - a. sửa các phương thức bị ảnh hưởng bởi thay đổi của Node.
  - b. thêm hằng thành viên (const data member) *maxSize* để giới hạn kích thước ngăn xếp, giá trị này được người dùng cung cấp khi khai báo một đối tượng ngăn xếp (chú ý constructor)
  - c. thêm phương thức *isFull()* để kiểm tra xem ngăn xếp đã đầy hay chưa.
  - d. trong các phương thức hiện có, hãy sửa nội dung các phương thức có liên quan đến kích thước tối đa của ngăn xếp, chẳng hạn *push()*

e. bổ sung copy constructor cho lớp Stack.

- ❖ Chỉnh lại/bổ sung các chú thích trong các file mã nguồn để phù hợp các thay đổi đã thực hiện

**Bài 3:** Cài đặt các lớp Vecto, Matrix, lớp đa thức **Polynomial**. Định nghĩa các phép toán  $+$ ,  $-$ ,  $*$ ,  $=$ ,  $>>$ ,  $<<$ ,  $[]$  (truy xuất tọa độ của vecto hoặc hệ số của đa thức),  $()$  (truy xuất phần tử của ma trận hoặc để tính giá trị của đa thức),  $^$  (tính độ dài vecto hoặc tính giá trị định thức).

Viết chương trình minh họa.

*Chú thích:*

- Vecto gồm số chiều của vecto và một mảng động chứa tọa độ vecto.
- Matrix gồm số hàng, cột và một mảng động 2 chiều chứa nội dung ma trận.
- Lớp Polynomial gồm cấp của đa thức và một mảng động chứa hệ số đa thức.